Influencer Vetting App

How the Whole Thing Works

A plain-English guide — no coding knowledge needed

This document explains, in simple words, what the app does from start to finish, and then walks through every important file one by one — what it is, why it exists, and how it fits into the bigger picture.

Prepared as an interview-ready explainer

Contents

1 The Big Picture (say this in an interview)

1.1 What problem does it solve?

A company (OpenArt) runs a program where social-media creators *apply* to become paid partners. Hundreds or thousands of people fill in a form with links to their Instagram, TikTok, YouTube, and LinkedIn accounts. Someone has to check each applicant and decide: *is this person a real, big-enough influencer worth working with?*

Doing that by hand is slow and boring. This app does it automatically.

> **Simple analogy:** Think of it as an automatic bouncer at a club door. Everyone hands over their ID (their social links). The bouncer checks each one against the rules — big enough following? enough views? from an allowed country? — and only lets in the people who qualify, marking them in colour.

1.2 The flow, from start to finish

Here is the whole journey in plain words. This is the part you can say out loud in an interview:

1. **Upload.** The user opens a web page and uploads a spreadsheet (an Excel `.xlsx` file) full of applicants. Each row is one person, with columns for their YouTube, Instagram, TikTok, LinkedIn, and country.

2. **Pick a range.** The user can choose to check only some of the rows (for example rows 2 to 300) instead of all of them. This matters because some of the checks cost money per person, so you don't always want to run every row.

3. **Run the checks.** The user clicks "Run." For *each* row, the app does this in order:

   - **Country screen first** — if the person lives in an excluded country, stop immediately and don't waste any money checking them.
   - **Clean up the links** — turn messy inputs (like "@MyHandle" or "instagram.com/me/") into a tidy, standard form the app can actually look up.
   - **Check YouTube** — free, so it always runs. Does any of their videos have enough views?
   - **Check the paid platforms** — Instagram, then TikTok, then LinkedIn. These cost money, so as soon as *one* of them passes, the app stops checking the rest of the paid ones for that person (to save money).

4. **Decide a verdict.** Each platform gets a result: **PASS**, **FAIL**, **REVIEW** (can't tell automatically — a human should look), or **SKIPPED** (they're not on that platform). The person's overall verdict is **PASS** if they clear the bar on *at least one* platform.

5. **Show the winners.** The app displays only the people who passed, each row colour-coded by which platform qualified them (purple for Instagram, red for YouTube, black for TikTok, blue for

LinkedIn, yellow if more than one). A first column shows which platform they were approved on, as a clickable link straight to their profile.

6. **Download and remember.** The user can download those passing rows as a fresh Excel file. The run is also saved to an online history, so if they close the page and come back later, their past runs are still there.

One-sentence version: "It's a web app where you upload a spreadsheet of influencer applicants, it automatically checks each one's social-media accounts against follower and view thresholds, filters out excluded countries, and hands you back just the people who qualify — colour-coded and downloadable."

1.3 Two ways to run it

The exact same checking logic can be used in two ways:

- **The web app** (`app.py`) — the friendly point-and-click version most people use.
- **The command line** (`run.py`) — a "power user" version you run by typing a command, useful for large automated batches.

Both share the same "brain" (the `vetting/` folder). The user interface is just a shell around it.

1.4 The key idea: separation of "brain" and "face"

The most important design idea, and a great thing to mention in an interview:

The **decision-making logic** (the rules: how many followers is enough, how to clean a link, how to combine results) is kept completely separate from the **parts that talk to the outside world** (the website, the Excel files, the paid APIs).

Why this is smart: the rules can be tested on their own with fake data, without spending money on real API calls or needing a real spreadsheet. It also means you can swap the website for a command line without rewriting a single rule.

2 Every File, One by One

The project is split into small files, each with one job. Below, each file is explained the same way: *what it is*, in one line, then *what it does* in plain words.

Think of the files in three groups:

- **The two front doors** — `app.py` and `run.py`.
- **The brain** — everything in the `vetting/` folder.
- **The helpers** that talk to the outside world — the API clients and file readers.

2.1 app.py — the website

> **One line:** the web page users see — upload, run, view results, download.

This file builds the whole interactive website using a tool called **Streamlit** (a library that turns Python into a web app without needing separate web-design code).

What it handles:

- Shows the title, the styling (the purple/pink gradients), and an optional password gate so not just anyone can open it.
- Provides the **upload box** for the Excel file and a **toggle** to turn the paid checks on or off.
- Shows a **slider** to pick which spreadsheet rows to check.
- When "Run" is clicked, it builds the connections to the outside services (YouTube, ScrapeCreators, Bright Data) using secret keys, then hands the rows to the brain to do the actual checking, showing a progress bar.
- Displays the **results table** (only passing rows, colour-coded) and the **download button**.
- Manages the **history sidebar** — listing past runs, and letting you re-open or delete them.

> **Why one detail matters:** Streamlit re-runs the whole page every time you click anything. So the app carefully *remembers* the last run's results in a "session memory," otherwise your results would vanish the moment you clicked the download button.

2.2 run.py — the command-line version

> **One line:** the same checks, but run by typing a command instead of clicking.

This is for running big batches without a browser. You type something like "check rows 2 to 301 of this file, including the paid checks," and it writes out a new Excel file with every applicant

annotated (their follower counts, verdicts, and notes) and a tick-box ticked for the ones who passed.

It reads the same secret keys, uses the same brain, and supports extra options like adding your own country-exclusion rules on the fly.

2.3 vetting/config.py — the settings/rulebook

> **One line:** every "magic number" and rule lives here, in one place.

Instead of scattering important numbers throughout the code, they all sit here so they're easy to find and change. For example:

- How many followers count as "enough" (Instagram/TikTok: 1,000; LinkedIn: 800).
- How many views a YouTube video needs (100,000).
- The list of **excluded countries**.
- Which spreadsheet columns to **hide** from the final output (form junk, internal scoring, personal data).
- The policy that a person qualifies if they pass *any one* platform, and that the paid checks stop early once one passes (to save money).

> **Why this is good practice:** if the company later says "actually, make the YouTube bar 200,000 views," you change *one number in one file* — you never go hunting through the logic.

2.4 vetting/models.py — the vocabulary

> **One line:** defines the standard "shapes" of data everyone else uses.

This file doesn't *do* anything on its own; it defines the labels and containers the rest of the app passes around, so everyone speaks the same language. The important ones:

- **Verdict** — the five possible outcomes: PASS, FAIL, REVIEW, SKIPPED, EXCLUDED.
- **Handle** — a cleaned-up social account (the tidy username plus the proper profile URL, and whether it could be resolved).
- **PlatformResult** — the result for one platform for one person (follower count, average views, and the verdict).
- **RowResult** — everything computed for one applicant across all four platforms plus the overall verdict.

- **ExclusionRule** — a rule describing which country values to drop.

> **Analogy:** this is the "form template" everyone fills in. It guarantees that when one part of the app says "here's a result," every other part knows exactly what fields to expect.

2.5 vetting/normalize.py — the link cleaner

> **One line:** turns messy, human-typed links into tidy, standard ones.

People type their accounts in a hundred inconsistent ways: "@me", "www.instagram.com/me/", "MY_HANDLE", or just "yes" (junk). This file's whole job is to make sense of that mess for each platform and produce a clean result:

- A usable handle plus a proper profile link (status: **OK**), or
- Nothing there / junk (status: **MISSING**), or
- Present but too ambiguous to use automatically — e.g. a "link tree" page that could point anywhere (status: **UNRESOLVABLE**, meaning a human should check).

It also decides, from the country text, whether the person should be screened out. Crucially, this file never touches the internet — it just cleans text — so it's fast and easy to test.

2.6 vetting/metrics.py — the measuring stick

> **One line:** the pure math of "do these numbers pass the bar?"

Once we have follower counts and video views, this file applies the actual thresholds and gives a verdict. Two clever details worth mentioning:

- It uses the **median** (middle value) of recent videos, not the average, so one freak viral hit — or one flop — doesn't distort the picture.
- It only looks at **recent** videos and ignores brand-new ones that haven't had time to gather views, so newer creators aren't judged unfairly.
- If it genuinely can't tell (e.g. an account with no videos), it returns **REVIEW** rather than guessing a FAIL.

LinkedIn is judged on follower count alone, because LinkedIn post data is too unreliable across sources to trust.

2.7 vetting/exclusions.py — the country gate

One line: checks a person's country against the exclusion rules.

A tiny file. It takes the country value and the list of rules, and returns the first rule that "fires" (matches), or nothing if the person is allowed through. It runs *first* in the pipeline so no money is ever spent checking someone who was going to be excluded anyway.

2.8 vetting/pipeline.py — the conductor

One line: runs the full check for one person, in the right order.

If the other files are musicians, this is the **conductor** that tells them when to play. For a single applicant it:

1. Runs the country screen — and stops right there if excluded.
2. Cleans all four links (via `normalize.py`).
3. Checks YouTube (free, always).
4. Checks the paid platforms in order — Instagram, TikTok, LinkedIn — stopping early once one passes.
5. Combines everything into one overall verdict.

Key design point: any of the outside connections can be "switched off" (set to nothing). If, say, there's no LinkedIn key, that step is simply skipped instead of crashing. This is what lets you do a free "dry run" without paying for anything.

2.9 vetting/youtube.py — the YouTube checker

One line: asks YouTube's official (free) service for a channel's video views.

It looks up the channel, finds its recent uploads, and checks whether any single video is above the view threshold. It deliberately uses the "cheap" parts of YouTube's service to stay within the free usage limits. If YouTube can't be reached or the channel isn't found, it returns **REVIEW** instead of failing the whole run.

2.10 vetting/scrapecreators.py — the Instagram & TikTok checker

One line: the paid service that fetches Instagram and TikTok numbers.

Instagram and TikTok don't give this data away freely, so the app pays a service called **ScrapeCreators** to fetch follower counts and recent video views. This file knows the exact shape of that service's responses and turns them into the app's standard **PlatformResult**. If the account is deleted, private, or the response is broken, it safely returns **REVIEW** rather than crashing.

2.11 vetting/brightdata.py — the LinkedIn checker

> **One line:** a different paid service, used only for LinkedIn followers.

LinkedIn is hard to read. The earlier service (ScrapeCreators) came back empty about 89% of the time on LinkedIn, so LinkedIn was moved to a stronger service called **Bright Data**, which recovers most profiles. This file sends one profile link, reads back the follower count, and judges it against the LinkedIn bar (800 followers). Same safety net: anything unexpected becomes **REVIEW**.

2.12 vetting/frames.py — the results builder & colourer

> **One line:** turns raw results into the pretty, coloured table and the downloadable Excel file.

This is where the output people actually see gets built. It:

- Figures out which spreadsheet columns are which (even if their order or exact names differ).
- Keeps *only* the passing rows.
- Picks each row's colour based on which platform passed (and yellow for multiple).
- Adds the leading **"Approved"** column (a clickable link to the qualifying profile) and a **"Row"** column showing the original spreadsheet row number.
- Hides the junk/personal columns listed in the config.
- Builds both the on-screen coloured table *and* the matching downloadable Excel workbook.

2.13 vetting/sheet_io.py — the Excel reader/writer (CLI)

> **One line:** reads rows from and writes results back into Excel, for the command-line version.

The command-line tool needs to open the original workbook, read the right columns, and later write all the computed numbers and verdicts back into new columns of a copy — ticking a checkbox for the clear passes. This file handles all that Excel plumbing, kept separate from the logic so the logic stays clean and testable.

2.14 vetting/history.py — the memory (past runs)

One line: saves each run online so it survives closing the browser.

Uses an online database service called **Supabase**. Every finished run is stored two ways: the full result (so it can be re-shown exactly) and a small summary row (date, how many were vetted, how many passed) that powers the fast history list in the sidebar. It's completely optional — if Supabase isn't set up, the app just runs without saving history.

The whole system in one breath: `app.py`/`run.py` take the input → `pipeline.py` conducts the check for each row → `exclusions.py`, `normalize.py`, `metrics.py` apply the rules → `youtube.py`, `scrapecreators.py`, `brightdata.py` fetch the real numbers → `frames.py` paints the results → `history.py` remembers them. And `config.py` + `models.py` quietly hold the settings and the shared vocabulary underneath it all.

3 The Journey of One Record

This section follows *one single applicant* through every file, in order, so you can see exactly how the files hand work to each other. It's the "end to end" story of one row.

3.1 Meet our example applicant

Let's say row 42 of the uploaded spreadsheet contains a person we'll call **Jane**:

YouTube:	`@janedoe`
Instagram:	`www.instagram.com/janedoe/`
TikTok:	`(blank)`
LinkedIn:	`(blank)`
Country:	`Spain`

Now watch what each file does to Jane's record, one hop at a time.

3.2 Hop 1 — `app.py`: the record enters the system

The user uploads the spreadsheet and clicks "Run." `app.py`:

- Reads the whole Excel file into a table in memory.
- Works out which column is which (asks `frames.py` to match "YouTube," "Instagram," etc. to the real headers).
- Sees the row slider covers row 42, so Jane is included.
- Builds the outside connections (YouTube, ScrapeCreators, Bright Data) using the secret keys.
- Hands the selected rows over to `frames.py` to be processed.

Jane's record right now: still raw, messy text — exactly as typed.

3.3 Hop 2 — `frames.py`: one row is prepared and passed on

`frames.py` goes row by row. For Jane it:

- Tags her with her original position (**row index 42**) so she can be matched back to the spreadsheet later.
- Converts any empty cells (her blank TikTok and LinkedIn) from spreadsheet-"empty" into a clean "nothing here."
- Passes Jane's record to `pipeline.py` — the conductor — to actually run the checks.

Jane's record right now: a tidy package with her four links, her country, and her row number.

3.4 Hop 3 — `pipeline.py`: the conductor takes over

This is the heart of the journey. `pipeline.py` runs these steps *in this exact order* for Jane:

Step 3a — Country screen (asks `exclusions.py`). It sends "Spain" to `exclusions.py`, which checks it against the excluded list in `config.py`. Spain is *not* excluded, so Jane continues. (If she'd said "India," the story would **end here** — verdict **EXCLUDED**, and not a cent spent checking her socials.)

Step 3b — Clean the links (asks `normalize.py`). Each of Jane's four links is sent to `normalize.py`:

- `@janedoe` (YouTube) → a clean YouTube reference, status **OK**.
- `www.instagram.com/janedoe/` → handle `janedoe` + proper URL, status **OK**.
- blank TikTok → status **MISSING**.
- blank LinkedIn → status **MISSING**.

The proper profile URLs are saved so the results table can link to them later.

Step 3c — Check YouTube (asks `youtube.py`). Because YouTube is free, it always runs. See Hop 4.

Step 3d — Check the paid platforms, in order. Instagram first (Hop 5), then TikTok, then LinkedIn. But TikTok and LinkedIn are **MISSING** for Jane, so they're marked **SKIPPED** without any API call. And thanks to the money-saving rule, if Instagram *passes*, the app wouldn't bother with the rest anyway.

Step 3e — Combine (uses the rule in `config.py`). It gathers all four platform results and decides the overall verdict. Because the policy is "pass on *any one* platform," a single PASS makes Jane an overall **PASS**.

3.5 Hop 4 — `youtube.py`: the free check runs

`youtube.py` takes Jane's clean YouTube reference and asks YouTube's official service:

- Find her channel → find her recent uploads → read each video's view count.
- Does any video clear 100,000 views? Say her best video has 40,000. That's below the bar → verdict **FAIL** for YouTube.

It hands back a neat **PlatformResult** (the shape defined in `models.py`) saying: handle janedoe, best views 40,000, verdict FAIL.

3.6 Hop 5 — `scrapecreators.py` + `metrics.py`: the paid check

For Instagram, `pipeline.py` calls `scrapecreators.py`:

- It pays the ScrapeCreators service and gets back Jane's Instagram data: say **8,500 followers** and her recent videos' view counts.
- It cleans up that raw response and passes the numbers to `metrics.py` — the measuring stick.

`metrics.py` then applies the rules:

- 8,500 followers is above the 1,000 bar. Good so far.
- It takes the **median** of her recent video views — say 6,000 — which is above the 3,000 view bar.
- Both bars cleared → verdict **PASS** for Instagram.

Back comes another **PlatformResult**: handle janedoe, 8,500 followers, 6,000 avg views, verdict PASS.

> **Note:** LinkedIn would use `brightdata.py` instead of ScrapeCreators here — same idea, different service — but Jane has no LinkedIn, so that file never gets involved for her.

3.7 Hop 6 — back in `pipeline.py`: the final verdict

Now the conductor has all four results for Jane:

```
YouTube:    FAIL (40,000 views)
Instagram:  PASS (8,500 followers)
TikTok:     SKIPPED (no account)
LinkedIn:   SKIPPED (no account)
```

Rule: pass on *any one* platform. Instagram passed → **overall verdict = PASS**. Jane is a qualified applicant.

3.8 Hop 7 — `frames.py` again: Jane becomes a coloured row

The finished results for every applicant come back to `frames.py`, which builds the output:

- Jane passed, so she's **kept** (people who failed are dropped from the view).
- Only Instagram passed, so her row is coloured **purple**.
- An "Approved" column is added with a clickable link to her Instagram.
- A "Row" column shows **42** — her original spreadsheet row.
- The junk/personal columns are stripped out.

It builds both the on-screen coloured table *and* a matching downloadable Excel file.

3.9 Hop 8 — `history.py`: the run is remembered

Finally, `app.py` asks `history.py` to save the whole run to the online database, so even after the browser closes, this run — including Jane's purple passing row — can be re-opened later from the sidebar.

3.10 The one-record flow at a glance

Jane's raw row
→ `app.py` (read file, build connections)
→ `frames.py` (tag with row 42, tidy cells)
→ `pipeline.py` (conduct the checks)
→ `exclusions.py` (Spain? allowed ✓)
→ `normalize.py` (clean the 4 links)
→ `youtube.py` (YouTube: FAIL)
→ `scrapecreators.py` + `metrics.py` (Instagram: PASS)
→ combine ⇒ overall **PASS**
→ `frames.py` (purple row, Approved link, row 42)
→ `history.py` (saved online)
→ **shown to the user + downloadable**

(`config.py` supplied every threshold and rule along the way; `models.py` supplied the shared "result" shapes passed between files.)

4 What If the API Credits Run Out Mid-Run?

This is a real risk worth understanding — and the app has a specific safeguard for it. It's also a great thing to talk about in an interview, because it shows you thought about failure, not just the happy path.

4.1 The problem (what would happen without the safeguard)

When an API runs out of credits it doesn't return data — it returns an error code. The important ones are:

- **402 Payment Required** — you're out of credits.
- **429 Too Many Requests** — you're being rate-limited.
- **401 / 403** — your key is rejected or over quota.

Every checker safely catches errors so one bad profile can't crash a run of 3,000 people. But that safety net created a nastier problem:

> **The silent-failure trap:** if the credits died at row 50 of 500, every remaining person would be marked **REVIEW**. **REVIEW** isn't **PASS**, so they'd all quietly *disappear* from the results table. The run would show a green progress bar, finish "successfully," and just look like a run where fewer people qualified. **You'd have no idea 450 people were never actually checked.**

4.2 The safeguard (what happens now)

A new file, `vetting/quota.py`, acts as a **watchdog**. It works like this:

1. Each checker now records *why* a lookup failed — specifically the error code the API gave back.
2. The watchdog tells apart two very different failures:
   - "**This profile is private/deleted**" (a 404) — normal, keep going.
   - "**Your plan is the problem**" (401/402/403/429) — dangerous.
3. It counts these plan-failures **consecutively, per platform**. If Instagram fails this way **3 times in a row**, the run stops immediately.
4. Because it counts *consecutive* failures, one random blip doesn't stop anything — a single successful lookup resets the counter. Only a real, persistent outage triggers the abort.
5. Counting is **per platform**, so a dead Instagram key doesn't stop the run because of unrelated LinkedIn hiccups.

4.3 The crucial part: nothing already earned is lost

When the watchdog stops the run, it doesn't throw the work away. It carries all the results gathered so far with it, so:

- A clear red banner appears: *"Run stopped early — API credits exhausted"*, naming the platform and the error code.
- **Everyone who already passed is still shown**, still colour-coded, still with their clickable profile links.
- **The download button still works** — you get the Excel file of the passers found before the credits ran out.
- The run is saved to history marked **"(partial)"** so you can tell later that it wasn't a complete pass.
- The banner tells you which row it stopped at, so you can top up the key and re-run from exactly there instead of starting over and paying twice.

4.4 A worked example

A real test of this behaviour: a 500-row run where the Instagram plan dies after the first two people.

Before the safeguard: all 500 rows get called. Rows 3–500 are all marked REVIEW. Table shows 2 passers. Looks like a normal, finished run.

With the safeguard:

- Rows 1–2 pass and are kept.
- Rows 3, 4, 5 fail with 402 → three in a row → **run stops at row 5**, not row 500.
- Red banner: *"instagram API returned HTTP 402 three times in a row — the key is out of credits…Stopped after 5 rows."*
- The 2 passing rows are still displayed and downloadable.

4.5 Why stopping early is itself valuable

Two benefits beyond honesty:

- **It saves time.** It stopped after 5 rows instead of pointlessly hammering a dead API 500 times.
- **It protects the money-saving logic.** Normally the app skips the remaining paid platforms once one passes. But when everything errors, nothing ever "passes," so it would keep calling *all three* paid services for every remaining row — the most wasteful possible behaviour, exactly when something is already wrong.

4.6 The command-line version too

`run.py` gets the same protection: it stops, writes the partial results to the Excel file anyway, logs a loud error saying how many rows were actually done, and exits with a **non-zero exit code** —

which is the standard way to tell an automated script “this job did not fully succeed.”

End of guide.